

Cours 30 — Au-delà de Python · Lancer un projet

Semaine 30 ■ Manuel Hachette ch. 14 (§ 14.1, 14.2) ■ 2 × 1 h

Objectifs

- Comparer les langages de programmation : propriétés et usages
- Comprendre pourquoi il existe des milliers de langages
- Savoir démarrer un projet logiciel (méthode du projet final)

1. Des langages différents... (cours A)

Vous en connaissez déjà quatre : Python, l'assembleur (S21), HTML/CSS (S26 — des langages de *description*), JavaScript (S27). Quelques propriétés qui les distinguent :

- **Compilé ou interprété** : traduit une fois pour toutes en langage machine (C, exécutables rapides) ou traduit au fil de l'exécution (Python — souple, plus lent ; d'où nos mesures des semaines 16-17).
- **Typage** : Python vérifie les types à l'exécution (le `TypeError` découvert en S4) ; d'autres langages les vérifient *avant* (à la compilation), attrapant des bugs plus tôt.
- **Niveau d'abstraction** : de l'assembleur (chaque instruction machine) à Python (une ligne = des milliers d'instructions).
- **Paradigme** : impératif (nos programmes : instructions qui modifient un état), fonctionnel, orienté objet (Terminale!)...

2. ...pour des usages différents

usage	langages typiques	pourquoi
science des données, enseignement	Python	expressif, bibliothèques
systèmes, jeux, embarqué	C, C++, Rust	vitesse, contrôle mémoire
Web côté client	JavaScript	le seul du navigateur
applications mobiles	Kotlin, Swift	liés aux plateformes

Aucun langage n'est « le meilleur » : chacun optimise un compromis (vitesse d'exécution, vitesse d'écriture, sûreté, écosystème). Un informaticien apprend des langages toute sa vie — et le deuxième coûte dix fois moins que le premier : vous l'avez vécu avec JavaScript, où variables, conditions, boucles et fonctions vous attendaient déjà.

3. Démarrer un projet (cours B — méthode)

Le projet final commence cette semaine (sujet distribué à part). La méthode, condensé de l'année :

1. **Spécifier** (S13) : que fait le programme ? qui l'utilise ? Écrire trois *scénarios d'utilisation* concrets (« l'utilisateur lance, saisit..., obtient... »).
2. **Découper** (S3) : la liste des fonctions, leurs docstrings — avant leur code. Structures de données choisies et justifiées (S8-S10).
3. **Prioriser** : le *cœur minimal qui marche* d'abord (règle des mini-projets 1 et 2) ; les bonus en fin de liste.

4. **Itérer** : coder petit, tester (S13), committer une version qui marche avant d'attaquer la suite.
5. **Se répartir** (binômes) : par fonctions, avec les docstrings pour contrat d'interface — et chacun teste les fonctions de l'autre.

Les trois pièges mortels du projet, vus cent fois : commencer par l'accessoire (l'écran-titre...), tout écrire d'un bloc sans jamais exécuter, découvrir la veille que « ça ne marche plus depuis 15 jours ». L'antidote est le même : **toujours avoir une version qui tourne**.

À retenir

- Compilé/interprété, typage, abstraction, paradigme : les axes qui distinguent les langages.
- Le langage se choisit selon l'usage ; les concepts, eux, se transportent (Python → JavaScript en une semaine).
- Projet : spécifier, découper, prioriser le cœur, itérer en gardant une version qui marche.